STATEMEN PERULANGAN

Jurusan Teknik Elektro
Prodi. Teknik Telekomunikasi D4
2022
@teamAlgo

AGENDA

- Pendahuluan
- Statemen *do-while*
- Statemen *While*
- Statemen for
- Iterasi di dalam iterasi
- Statemen Break
- Statemen continue

PENDAHULUAN

- Mengulang suatu proses merupakan tindakan yang banyak dijumpai dalam pemrograman
- Pada semua bahasa pemrograman, pengulangan proses ditangani dengan suatu mekanisme yang disebut *loop*
- Dengan menggunakan *loop*, suatu proses yang berulang dapat diimplemetnasikan dengan **baris program yang pendek**

PENDAHULUAN (CONT.)

- Pada bahasa C terdapat tiga struktur loop, yaitu dengan menggunakan statemen *while*, *do-while*, dan *for*
- Terkait dengan penanganan loop, terdapat beberapa pernyataan lain yang berkaitan diantaranya adalah pernyataan yang digunakan untuk keluar dari loop, seperti *break*, *goto*, dan *continue*

STATEMENT *DO-WHILE*

- Statemen do-while digunakan untuk mengulang suatu proses selama kondisinya terpenuhi
- Bentuk sintaks statemen *do-while* adalah:

```
do
    Pernyataan
while (kondisi)
```

STATEMENT *DO-WHILE* (CONT.)

- Jika nilai kondisi benar, maka pernyataan akan dieksekusi kembali, begitu pun untuk pemeriksaan kondisi akan diulangi kembali, dan begitu seterusnya, hingga nilai kondisi dinyatakan salah (false)

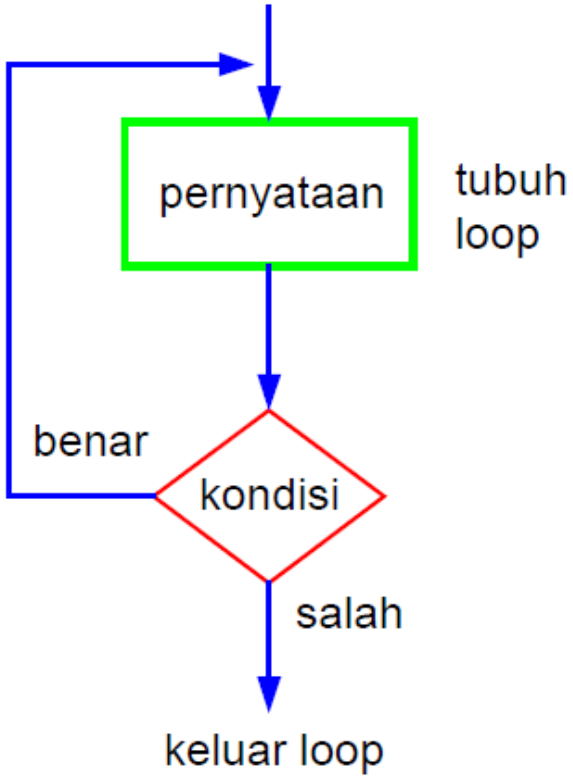
STATEMENT *DO-WHILE* (CONT.)

- Pada pernyataan do-while, tubuh loop dapat berupa pernyataan tunggal, pernyataan majemuk maupun pernyataan kosong

- Pada statemen *do*, awalnya pernyataan akan dieksekusi tanpa ada proses pengujian, selanjutnya nilai kondisi akan diuji.

STATEMENT *DO-WHILE* (CONT.)

- Diagram alir statement *do-while* :



STATEMENT *DO-WHILE* (CONT.)

```
1.  /* ------------------------------------*/
2.  /* File Program : TurboC.C */
3.  /* Menampilkan tulisan Turbo C */
4.  /* sepuluh kali */
5.  /* ------------------------------------*/
6.
7.  #include <stdio.h>
8.  main ()
9.  {
10.             char pencacah;
11.             pencacah = 0;
12.             do
13.             {
14.                     puts("Turbo C++\n");
15.                     pencacah++;
16.             }
17.             while (pencacah < 10);
18. }
```

STATEMENT *WHILE*

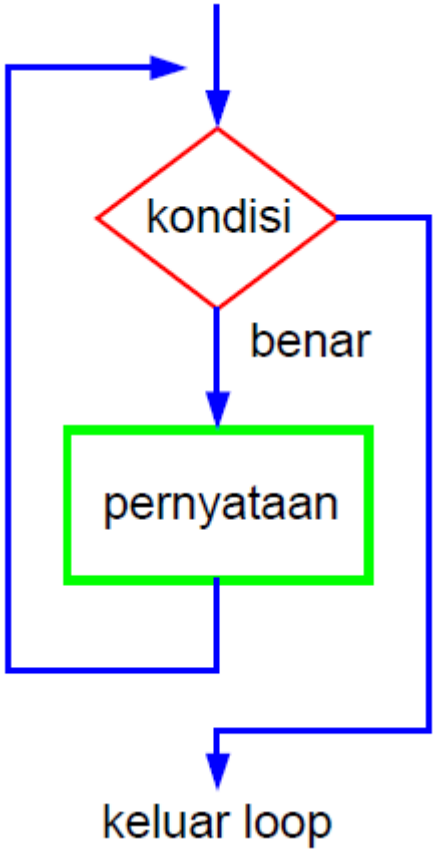
- Berbeda dengan statement *do-while*, pada statemen *while* pengujian terhadap *loop* dilakukan di bagian awal (sebelum tubuh *loop*). Bentuk sintaksisnya adalah sebagai berikut:

  while (kondisi)
  pernyataan

- Pernyataan dalam loop dapat berupa pernyataan tunggal, majemuk maupun kosong.

STATEMENT *WHILE* (CONT.)

- Diagram alir statement *while* :



STATEMENT *WHILE* (CONT.)

- Pada statement *while* kemungkinan pernyataan yang merupakan tubuh loop memungkinkan untuk tidak dijalankan sama sekali, yaitu jika pada pengujian kondisi while yang pertama kali ternyata kondisi bernilai salah
- Perhatikan program berikut ini, bagaimanakah hasil *output* dari program tersebut.

STATEMENT *WHILE* (CONT.)

```
1.  #include <stdio.h>
2.  main( )
3.  {
4.      int i, jum;
5.
6.      i = 5; jum = 0;
7.      while (i--)
8.      {
9.          printf("%d ",i);
10.         jum = jum + i;
11.     }
12.     printf("%d", jum);
13. }
```

LATIHAN

- Buatlah algoritma dan program untuk menampilkan deret berikut :

  20 24 28 32 36 42 48 52 56 . . . 200.

  dengan menggunakan

  a. while
  b. do-while

STATEMENT *FOR*

- Statemen for digunakan untuk mengulang eksekusi suatu pernyataan dengan jumlah pengulangan yang sudah ditentukan
- Bentuk sintaksisnya adalah sebagai berikut:

For (kondisi_1; kondisi_2; kondisi_3)

pernyataan;

- Pernyataan dalam loop dapat berupa pernyataan tunggal, majemuk maupun kosong.

STATEMENT *FOR* (CONT.)

- Fungsi dari masing-masing ungkapan pada statemen *for* adalah:
  - kondisi_1 : digunakan untuk memberikan inisialisasi terhadap variabel pengendali loop.
  - kondisi_2 : digunakan sebagai kondisi untuk keluar dari loop.
  - kondisi_3 : digunakan sebagai pengatur kenaikan nilai variabel pencacah/pengendali loop.
- Ketiga kondisi dalam *for* tersebut dipisahkan dengan dua tanda titik koma (……; …..; ……)

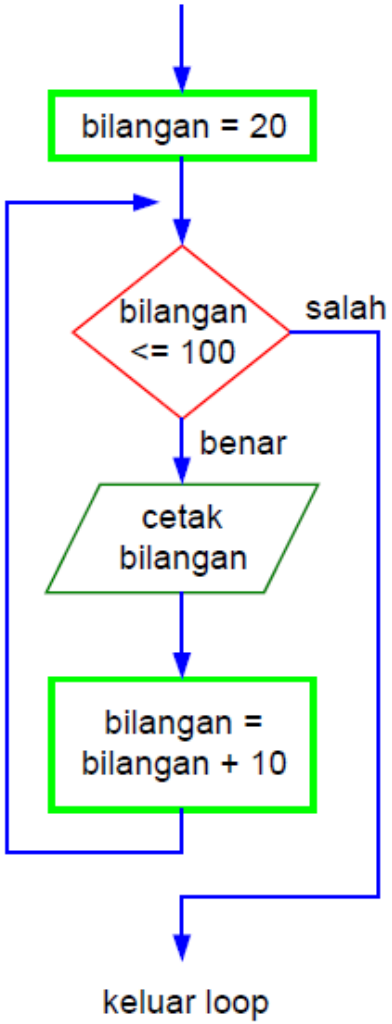
STATEMENT *FOR* (CONT.)

- Loop *for* sebenarnya identik dengan bentuk loop *while* sebagai berikut:

```
kondisi_1;
while (kondisi_2)
{
        pernyataan
        kondisi_3;
}
```

STATEMENT *FOR* (CONT.)

- Diagram Alir Statemen *for*



STATEMENT *FOR* (CONT.)

```
1.  #include <stdio.h>
2.  main ()
3.  {
4.     int bilangan;
5.
6.     for(bilangan = 20; bilangan <= 100; bilangan +=10)
7.          printf("%d\n", bilangan);
8.  }
```

LATIHAN

- Buatlah algoritma ~~dan program~~ untuk menampilkan deret berikut :

  20 24 28 32 36 42 48 52 56 . . . 200.

  dengan menggunakan

  a. while
  b. do-while

LATIHAN

- Buatlah algoritma ~~dan program~~ untuk menampilkan semua bilangan genap yang terletak antara 20 sampai 100.
- Sama Seperti soal sebelumnya, namun berlaku untuk semua bilangan ganjil dan habis dibagi 3.

LOOP YANG TIDAK PERNAH BERHENTI (INFINITE LOOP)

- Loop yang tidak pernah berhenti (berputar terus) sering terjadi disebabkan adanya kesalahan pada penanganan kondisi yang digunakan untuk keluar dari loop.

- Sebagai contoh, jika ditulis:

```
for(bilangan = 80; bilangan >= 10; bilangan -= 0)
   printf("%d\n", bilangan);
```

INFINITE LOOP (CONT.)

- statemen di atas tidak akan pernah berhenti berputar disebabkan kondisi yang tidak pernah tercapai.

- Hal ini terjadi karena ungkapan ketiga dalam ***for*** tidak tepat. Seharusnya berupa:

  bilangan -= 10;

- Kesalahan yang tertulis adalah:

  bilangan -= 0;

INFINITE LOOP (CONT.)

- Oleh karena kondisi **bilangan >= 10 selalu bernilai benar (sebab bilangan tidak akan pernah berubah yaitu selalu bernilai 80) , maka pernyataan :**

  printf("%d\n", bilangan);

  akan dijalankan secara terus menerus

- kondisi di atas dapat dihentikan dengan menggunakan tombol CTRL-BREAK

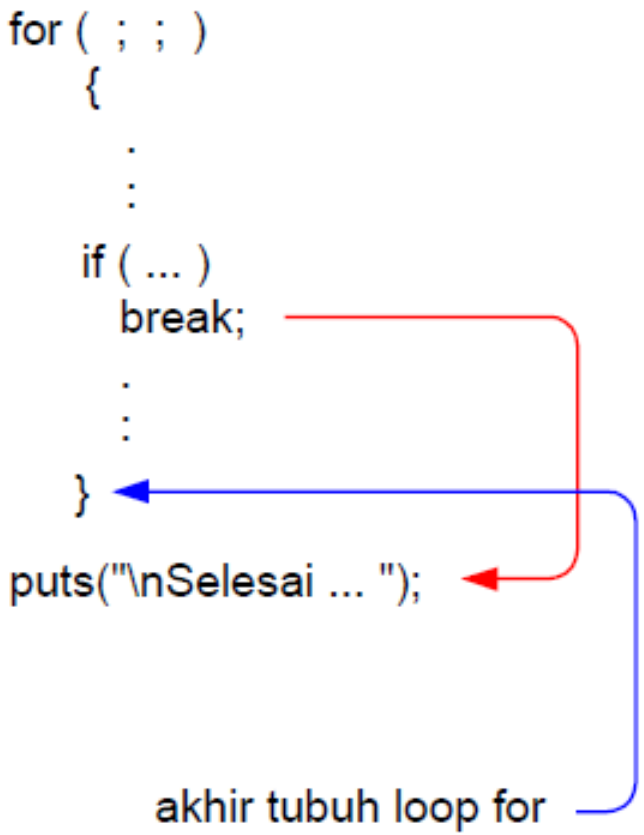
STATEMEN *BREAK*

- Statemen break berfungsi agar dapat keluar dari proses pengulangan (loop) *for*, *do-while*, dan *while*.

- Sedangkan pada statemen switch, break digunakan untuk menuju ke akhir (keluar dari) struktur switch.

STATEMEN *BREAK (CONT.)*

- penggunaan statement *break*



*Disamping ini memperlihatkan ilustrasi statemen **break** pada struktur **loop for**. Jika statemen **break** dijalankan (**logika bernilai positif / TRUE**) maka eksekusi program akan dilanjutkan ke statemen sesudah tubuh loop for (= **proses pengulangan berakhir**).*

STATEMEN *BREAK (CONT.)*

```
1.  /* ---------------------------------------*/
2.  /* File Program : Ketik.C */
3.  /* Pemakaian break pada loop for */
4.  /* ---------------------------------------*/
5.  #include <stdio.h>
6.  main ()
7.  {

8.       for(int i=0;i<=10 ; i++)
9.      {
10.      if (i == 9)
11.          break; /* keluar dari loop for */
12.        printf("cetak angka %d\n",i);
13.     }
14.       puts("\nSelesai ...");
15. }
```

Hasil Output :

```
cetak angka 0
cetak angka 1
cetak angka 2
cetak angka 3
cetak angka 4
cetak angka 5
cetak angka 6
cetak angka 7
cetak angka 8

Selesai ...
```

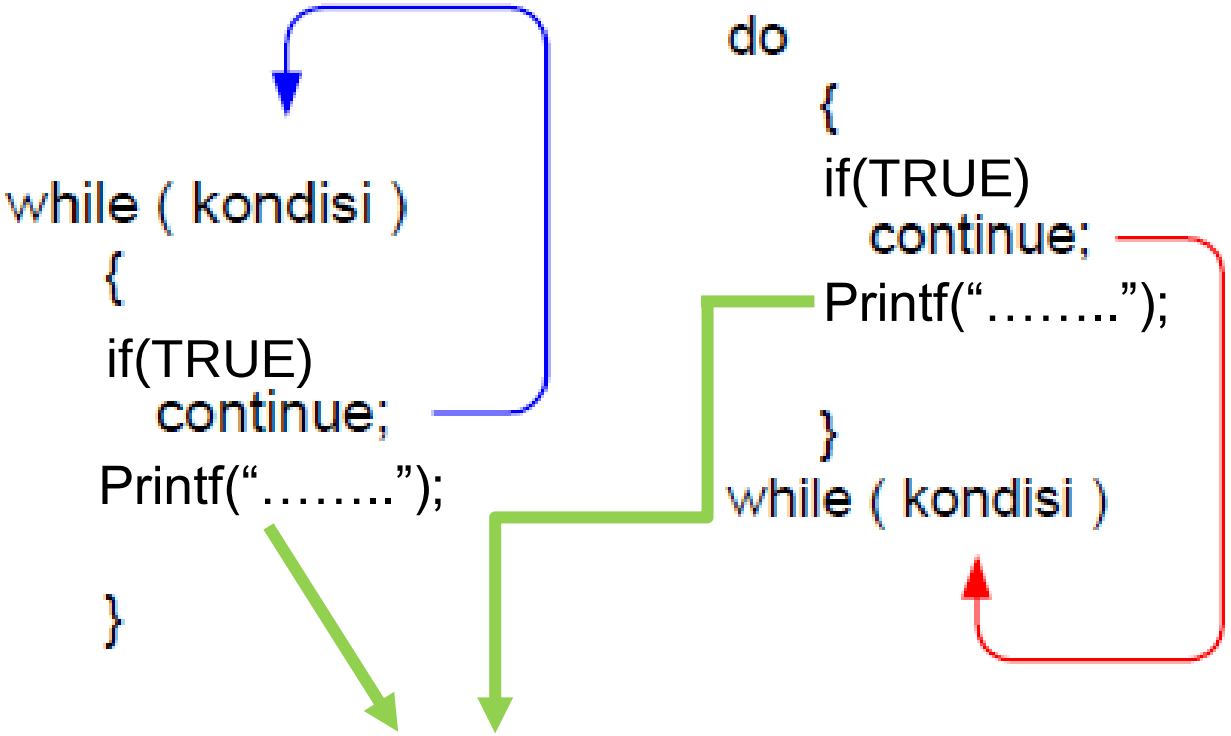
STATEMEN *CONTINUE*

- Pernyataan *continue* digunakan untuk mengarahkan eksekusi ke iterasi (proses) berikutnya pada *loop yang sama*

- Pada *do-while* dan *while*, pernyataan *continue* menyebabkan eksekusi akan menuju ke kondisi pengujian pengulangan *selanjutnya tanpa melaksanakan instruksi dibawahnya (instruksi dibawah pernyataan continue)*, seperti yang diperlihatkan pada diagram alir berikut.

STATEMEN *CONTINUE (CONT.)*

- Implementasi pernyataan continue



Jika pernyataan *continue* dijalankan maka pernyataan printf(“……”) tidak akan dilaksanakan.

STATEMEN *CONTINUE (CONT.)*

- Pada *loop for*, pernyataan *continue* akan menyebabkan *pengendali loop* (kondisi_3) dilaksanakan, kemudian *pembatas loop (*kondisi_2) akan diuji kembali.

- Berikut ini format *loop for* :

  for (kondisi_1; kondisi_2; kondisi_3)

STATEMEN *CONTINUE (CONT.)*

```
1.  /* File Program : Deret.C */
2.  /* Pemakaian continue pada for */
3.  #include <stdio.h>
4.  main ()
5.  {
6.      int x;

7.      for(x = 7;x <= 25 ; x += 2)
8.      {
9.          if (x == 15)
10.           continue;
11.         printf("%d ", x);
12.     }
13. }
```

Hasil Output :

```
7 9 11 13 17 19 21 23 25
```

/* contoh disamping menunjukkan jika instruksi ***continue*** dilaksanakan maka akan melanjutkan ke iterasi atau loop selanjutnya dan pernyataan printf("%d ", x); tidak akan dilaksanakan */

ARRAY

- Array/larik adalah kumpulan data dengan tipe serupa.
- KELEBIHAN ARRAY
  - kemampuannya menyimpan beragam data serupa dengan satu variabel.
  - elemen array dapat diakses dengan menggunakan indeksnya. Sehingga, waktu yang diperlukan untuk membaca sebuah array cenderung lebih singkat
  - memberikan lokalitas cache yang lebih baik

ARRAY

- KEKURANGAN ARRAY
  - *array* hanya bisa menyimpan data dengan tipe serupa. Jadi, data-data yang tidak memiliki kesamaan nilai tidak bisa disimpan dalam satu larik *array*.
  - Menggunakan memori yang lebih besar

CONTOH ARRAY

```
#include <iostream>
using namespace std;

const int MAX_ARRAY = 5;

int main ()
{
    string nama[MAX_ARRAY]={};

    for(int i=0;i<MAX_ARRAY;++i){
        cout<<"Masukan Nama : ";cin>>nama[i];
    }

    cout<<endl<<"=== Daftar Nama ==="<<endl;
    for(int i=0;i<MAX_ARRAY;++i){
        cout<<(i+1)<<". "<<nama[i]<<endl;
    }

    return 0;
}
```

CONTOH ARRAY

```
#include <iostream>

using namespace std;

int main()
{
  int nilai[5];

  nilai[0] = 23;
  nilai[1] = 50;
  nilai[2] = 34;
  nilai[3] = 78;
  nilai[4] = 90;

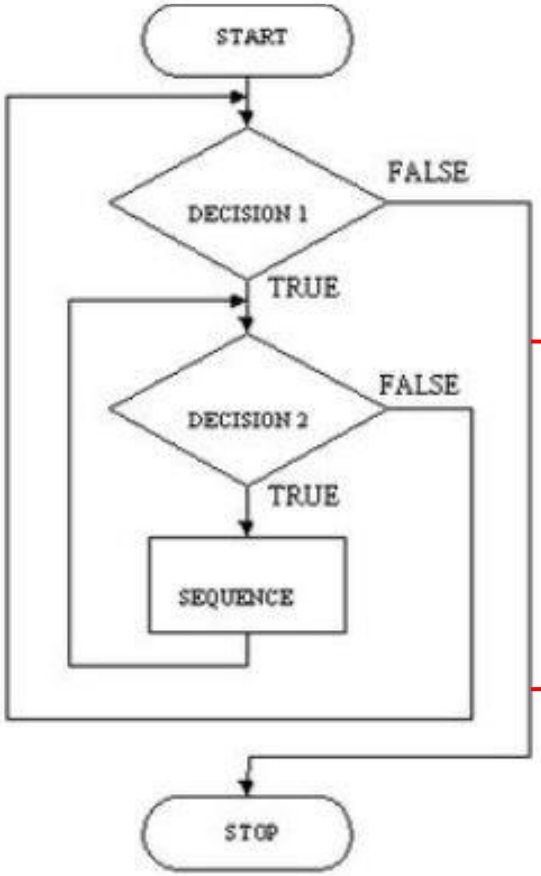
  cout << "Isi array pertama :" << nilai[0] <<endl;
  cout << "Isi array kedua   :" << nilai[1] <<endl;
  cout << "Isi array ketiga  :" << nilai[2] <<endl;
  cout << "Isi array keempat :" << nilai[3] <<endl;
  cout << "Isi array kelima  :" << nilai[4] <<endl;

  return 0;
}
```

ITERASI DI DALAM ITERASI

- Dalam suatu loop dapat terkandung loop yang lain.
- Penggunaan lebih dari 1 (satu) proses pengulangan atau iterasi (loop), umumnya bertujuan untuk membuat tabulasi data atau digunakan dalam penggunaan array multidimensi.
- Loop yang terletak di dalam loop umumnya disebut dengan *nested loop* (iterasi bersarang).

ITERASI DI DALAM ITERASI (CONT.)



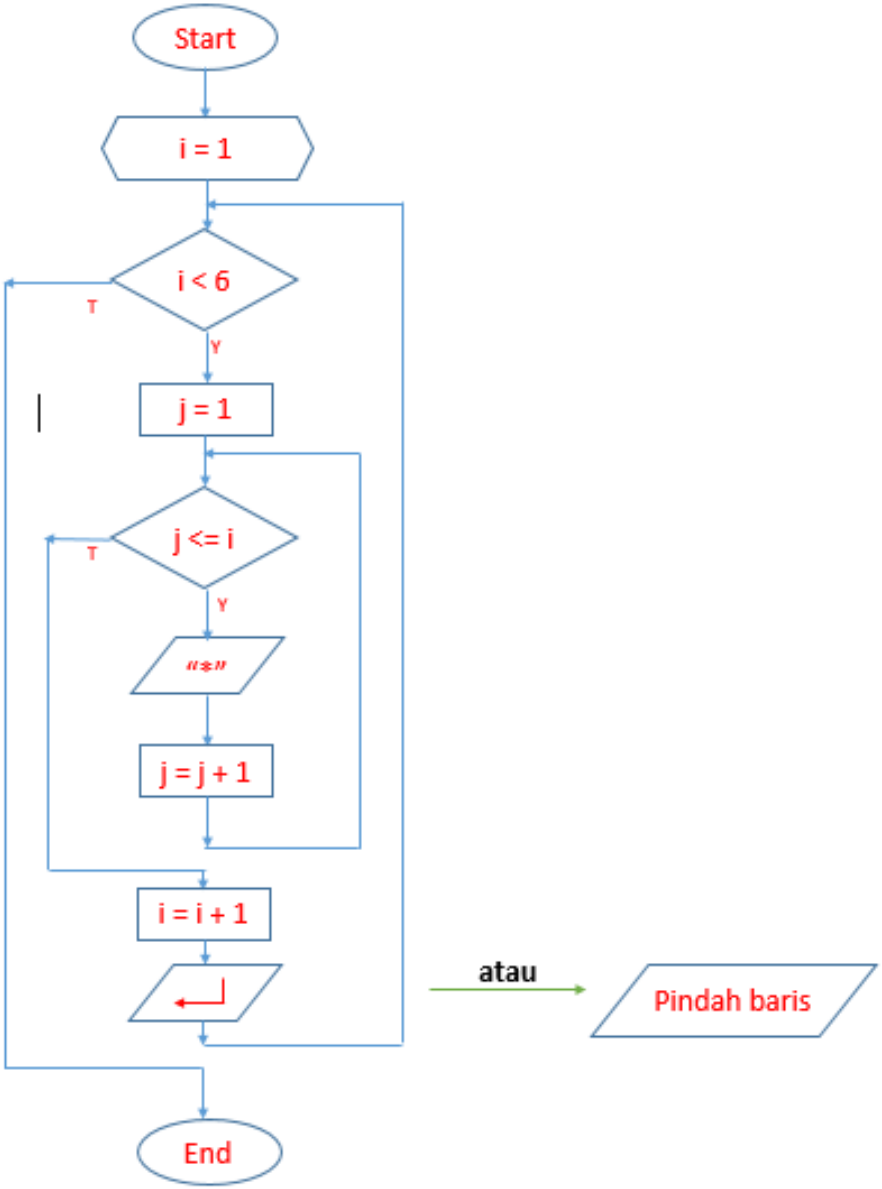
Sumber: https://www.pinterest.com/pin/499477414908915829/

Loop terdalam

Loop terluar

Loop terluar akan berakhir proses pengulangannya setelah loop terdalam berakhir terlebih dahulu.

ITERASI DI DALAM ITERASI (CONT.)



Algoritma tersebut digunakan untuk menampilkan hasil berikut ini :

```
*
* *
* * *
* * * *
* * * * *
```

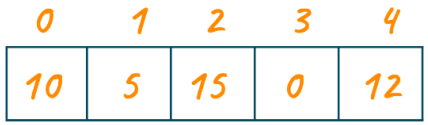
ALGORITMA BUBBLESORT

1. pengecekan dimulai dari elemen paling awal.
2. Elemen ke-1 dan ke-2 dari list dibandingkan.
3. Jika elemen pertama lebih besar dari elemen kedua, dilakukan pertukaran.
4. Langkah 2 dan 3 dilakukan lagi terhadap elemen kedua dan ketiga, seterusnya sampai elemen terakhir.
5. Bila sudah sampai di elemen terakhir dilakukan pengulangan lagi dari awal sampai tidak ada terjadi lagi pertukaran elemen.
6. Bila tidak ada pertukaran elemen lagi, maka elemen list terurut

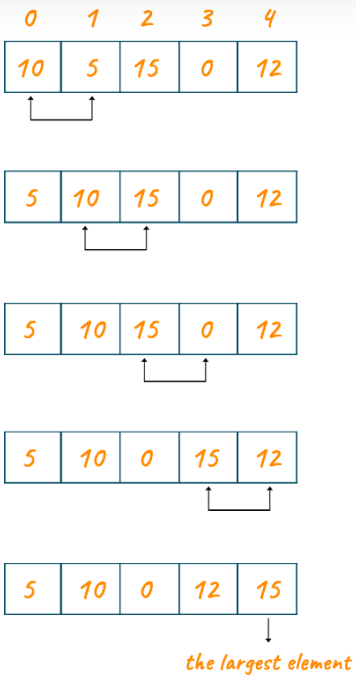
PSUEDOCODE UNTUK ALGORITMA *BUBBLE SORT*

```
procedure bubblesort( A : list of integer )
var
        temp,i : integer
        tukar :boolean
Algoritma
do
        tukar := false
        for i = 1 to length( A ) - 1 do
                if A[ i ] > A[ i + 1 ] then
                        temp:=A[i]
                        A[i]:=A[i+1]
                        A[i+1]:=temp
                        tukar := true
                end if
        end for

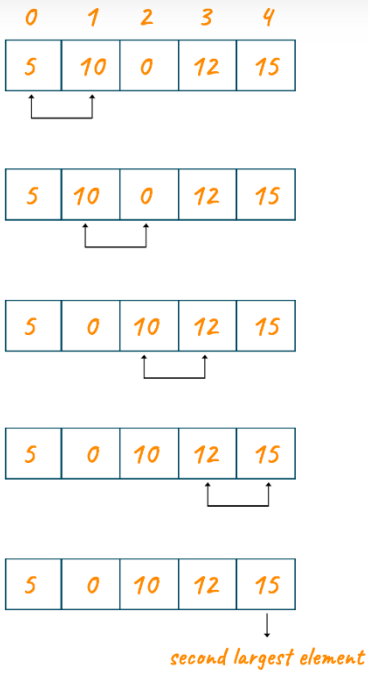
while tukar
```



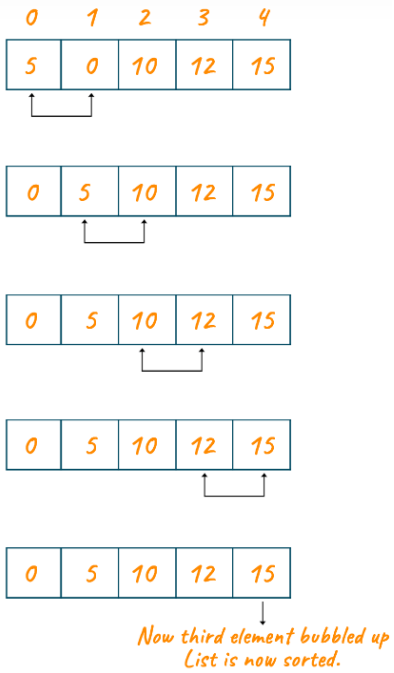
Pass 1:

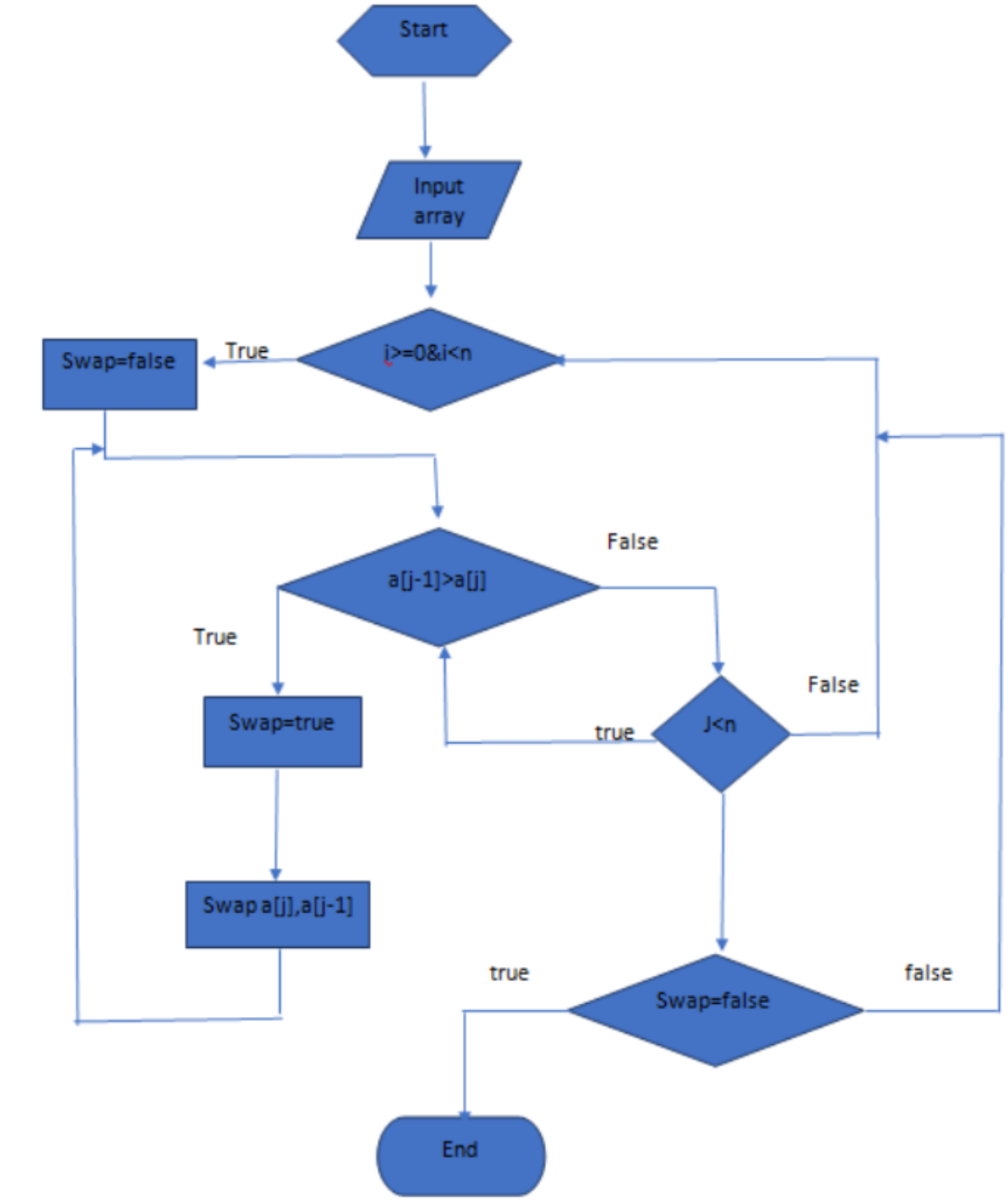


Pass 2:



Pass 3:




Start
Input array
i>=0&i<n
True
Swap=false
False
a[j-1]>a[j]
True
Swap=true
Swap a[j],a[j-1]
true
J<n
False
true
Swap=false
false
End